MCPGuardBench: Measuring System-Level Mitigations Against Tool-Access Attacks on MCP-Connected LLM Agents

Project code: G1-02 | **Seed:** G1 – Securing AI Agent Tool Access: A Benchmark for Model Context Protocol (MCP) Deployments

Supervisor: Muhammad Sajjad | **Instructor:** Muhammad Owais Idrees | AI4016 Agentic Artificial Intelligence, Fall 2026

Member	Roll number	Role
Zaid Bin Umer (group head)	23I-0671	Testbed, attack servers, defense gateway
Saud ur Rahman	23I-0540	Literature review, evaluation harness, experiments

GitHub repository: https://github.com/Zaid2004fast/MCP-GuardBench

Deliverable A – Literature Review & Research Gaps

A.1 How we selected the papers

To find the papers for this review, we searched IEEE Xplore, the ACM Digital Library, the ACL Anthology, OpenReview (ICLR), the AAAI proceedings, the NeurIPS and ICML proceedings, and the programmes of NDSS and ICSE. We looked for papers from 2025 and 2026 on three related topics: security benchmarks for MCP and other tool-using agents, attacks that target how an agent selects and reads its tools, and defenses that protect agents against prompt injection and misuse of their permissions.

We only kept a paper if it was peer-reviewed, was published in 2025 or 2026, had a clear method and an experimental evaluation, and was directly related to the security of tool access through MCP. Twelve papers met all of these conditions, and they are listed in Table A.2. Ten of them were published at venues ranked A* or A in the CORE ranking (ICLR, AAAI, NeurIPS, ICML, ACL, EMNLP, NDSS and ICSE).

We also cite some older foundational papers (ReAct [F1], AgentDojo [F2] and InjecAgent [F3]) and two well-known 2025 preprints (CaMeL [F4] and Progent [F5]) to give background. These are not counted toward the required ten papers.

A.2 Base paper

D. Zhang, Z. Li, X. Luo, X. Liu, P. Li and W. Xu, "MCP Security Bench (MSB): Benchmarking Attacks Against Model Context Protocol in LLM Agents," International Conference on Learning Representations (ICLR 2026). [1]

Proof of ranking. ICLR is ranked A* in the CORE2023 conference ranking, which can be checked on the ICORE portal at https://portal.core.edu.au/conf-ranks/2273/. The paper is part of the ICLR 2026 proceedings, and its poster page is https://iclr.cc/virtual/2026/poster/10007929.

Why we chose this paper. Out of all the papers we reviewed, MSB is the one that is closest to our project, for four reasons. First, it is the first benchmark that runs real MCP tools instead of simulated functions. Second, it organises the attacks according to the three stages an agent goes through when it uses a tool, which are planning which tool to use, calling the tool, and handling the tool's response. Third, it tests open-weight models such as Llama-3.1 8B and 70B, Llama-3.3 70B and Qwen3 8B and 30B, which we are also able to run. Fourth, its code and the definitions of its metrics are publicly available under the MIT license. Together, these mean that we can rebuild its attacks and metrics inside our own testbed and compare our results with the numbers it reports.

Problem, method, data and results. The paper asks how vulnerable MCP-connected agents are to attacks that a malicious MCP server can carry out at each stage of tool use.

- **Attacks:** The authors define 12 attack types. At the planning stage, the attacker can hide instructions in a tool's description, give a malicious tool the same name as a trusted one (name collision), or claim that a malicious tool should be preferred (preference manipulation). At the calling stage, the attacker can ask for extra parameters that leak private data. At the response stage, the attacker can pretend to be the user, return a fake error, redirect the agent to another tool, or inject instructions into retrieved data. The paper also tests mixtures of these attacks.
- **Benchmark:** The benchmark has 10 scenarios, 65 user tasks, 25 MCP servers, 304 benign tools and 405 attack tools. Combining these gives about 2,000 attack instances.
- **Agents:** The authors test 10 LLM agents that have access to a filesystem server and a desktop command server.

- **Metrics:** They measure the attack success rate (ASR), the performance under attack (PUA), which is the share of user tasks still completed during an attack, and the net resilient performance, NRP = PUA × (1 − ASR), which combines the two.

The main results are as follows:

- The average attack success rate across all models is 40.35%.
- Out-of-scope parameter attacks are the strongest, with an average success rate of 76.5% that reaches 98.75% on GPT-5 and Llama-3.3-70B.
- Models that are better at using tools are more vulnerable. For example, DeepSeek-V3.1 has an attack success rate of 60.94%, while Llama-3.1-8B has only 19.74%.
- The only defense the paper tests is the MCIP guardian classifier [4]. It lowers the attack success rate to 28.69%, but it also lowers the performance under attack from 56.50% to 48.91%, so the overall NRP barely changes (from 33.70% to 34.88%).

Limitations stated by the authors. The authors explain that detection-based defenses reject too many legitimate actions, and that blocking out-of-scope parameters also breaks normal tool calls. They suggest that future defenses need to be more aware of context and more dynamic.

Limitations we identified.

1. The paper evaluates only one defense, and that defense depends on a language model making the right judgement. The simpler system-level controls that real organisations use are not tested at all. These are allowing only approved tools (tool allowlisting), giving the agent only the permissions it needs (least-privilege credentials), asking a human to approve sensitive actions, and keeping an audit log.
2. The benchmark has no model of identity or permissions. The agent has unrestricted access to the filesystem and the shell, so the benchmark cannot tell the difference between an action the user is allowed to request and an action that only the agent's own credentials allow. Because of this, privilege escalation and confused-deputy attacks cannot be expressed in MSB.
3. Success is only judged for each attack instance. The paper does not measure how much human oversight a defense would need, such as how many approvals or false alarms it causes, and it does not measure whether an attack could be detected afterwards from logs.
4. The paper has some internal inconsistencies. It mentions 9 models in one place and 10 in another, and it says there are 12 attack types while listing 8 single attacks and 7 mixed ones. This encouraged us to make our own judging fully deterministic, so that every result can be re-run and checked exactly.

How our project builds on it. We rebuild MSB's attack taxonomy, its way of crossing user tasks with attacks, and its metrics inside a self-contained sandbox. On top of this, we add the parts that MSB is missing. These are an identity and credential model with two new attack classes (privilege escalation and confused deputy), a policy gateway that implements the four mitigations named in our project seed, and a comparison of those mitigations against MSB's own defense (an MCIP-style guard) on the same scenarios and on open-weight models that overlap with MSB's.

A.3 Critical summaries

[1] MSB, ICLR 2026 (base paper). This paper is summarised in Section A.2.

[2] Zong et al., "MCP-SafetyBench: A Benchmark for Safety Evaluation of Large Language Models with Real-World MCP Servers," ICLR 2026.

Problem: The authors point out that earlier MCP safety work either focuses on a single type of attack or uses simple toy servers that do not behave like real ones.

Method: They build a ReAct agent that carries out multi-turn tasks on live MCP servers. They define 20 attack types and group them by where the attack comes from. There are 11 server-side attacks, such as tool poisoning, tool shadowing, rug pulls and injection through function results. There are 4 host-side attacks, which are intent injection, data tampering, identity spoofing and replay. There are 5 user-side attacks, such as credential theft and misuse of excessive privileges.

Data: The benchmark contains 245 tasks over five domains (finance, navigation, repository management, browser automation and web search), built on top of MCP-Universe. The authors evaluate 13 models, four of which are open-weight.

Results: The attack success rate ranges from 29.8% for Qwen3-235B to 48.2% for o4-mini. Host-side attacks reach 81.9%, and identity spoofing works on all 13 models. Models that complete more tasks also tend to fail more often against attacks (the correlation is $r = -0.57$). The only defense tested is a safety prompt. It lowers the weighted attack success rate from 39.88% to 38.65%, which is not statistically significant, and it actually increases the attack success rate for all four open-weight models.

Limitations: Only one prompt-level defense is tested and no system-level controls are evaluated. Privilege misuse is modelled as something the user asks for in a prompt, rather than as a question of what the agent is authorised to do. The overall task success is also low (at most 21%), which makes it hard to study how defenses affect usefulness.

Relevance: We use this safety prompt as one of our baselines (B1). The paper also gives further evidence that prompt-level defenses are not enough, which supports our first research gap.

[3] Wang et al., "MCPTox: A Benchmark for Tool Poisoning on Real-World MCP Servers," AAAI 2026.

Problem: Most earlier work studies attacks that are injected through the outputs of tools. This paper instead studies attacks hidden in the metadata of a tool, such as its description, which the client loads when the tool is registered. This kind of attack works even if the poisoned tool is never called.

Method: The authors embed malicious instructions into tool descriptions using three attack templates. They generate the test cases with few-shot prompting and cover 10 categories of risk.

Data: The benchmark uses 45 live MCP servers, 353 real tools and 1,348 malicious test cases, and it evaluates 20 settings of LLM agents.

Results: The average attack success rate is about 36.5%, and the most vulnerable model is o1-mini at 72.8%. Again, more capable models are often more vulnerable, and even the model that refused the most (Claude-3.7-Sonnet) refused fewer than 3% of the attacks. The attacks make agents use legitimate tools to perform actions that were never authorised, and the models' safety training does not catch this.

Limitations: The paper only studies attacks and does not test any defenses. It mostly looks at single tool calls rather than long multi-step tasks, and it does not consider what permissions the agent actually has.

Relevance: This paper motivates our allowlist design, which checks a hash of each tool's description so that changed descriptions are noticed (first gap). Its observation that legitimate tools are misused for unauthorised actions is exactly the confused-deputy situation that we formalise in our second gap.

[4] Jing et al., "MCIP: Protecting MCP Safety via Model Contextual Integrity Protocol," EMNLP 2025.

Problem: The authors argue that it is difficult to analyse the safety of MCP because its clients and servers are separate and decentralised.

Method: They use the MAESTRO threat framework to analyse MCP and find two components that are missing from it, which are tools for tracking what happens and models that are aware of safety. They then propose MCIP, which records structured logs of information flows (who sent what information about whom to whom, and under which rule) and trains a guardian model on these logs.

Data: They define a taxonomy of 10 risk types plus a safe class. Their benchmark, MCIP-Bench, contains 2,218 dialogues built from Glaive and ToolACE, and they create 13,830 synthetic training examples. The guardian model is Llama-xLAM-2-8B fine-tuned on this data.

Results: The guardian improves risk-resistance accuracy from 13.35% to 54.16% and safety-awareness accuracy from 57.43% to 75.73%. However, its accuracy on normal function calling (the BFCL benchmark) drops from 72.04% to 65.79%.

Limitations: The evaluation is an offline classification task on synthetic logs, not a test against live attacks, and the authors themselves say that concrete attack techniques are not simulated. Risk resistance only reaches 54%. When MSB [1] used MCIP against live attacks, it reduced usefulness so much that the overall NRP hardly improved.

Relevance: This is the defense that our base paper evaluates, so we use an MCIP-style guard as our detection baseline (B2). Its idea of logging information flows also motivates our study of audit logs in the third gap.

[5] Zhang et al., "Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-based Agents," ICLR 2025.

Problem: Before this paper, there was no single framework that formally defined and compared attacks and defenses for LLM agents.

Method: The authors formally define direct and indirect prompt injection, memory poisoning, a new Plan-of-Thought backdoor attack and mixed attacks, and they compare 11 defenses against them. They also introduce the NRP metric, which MSB later adopted.

Data: The benchmark covers 10 scenarios (such as e-commerce, finance and autonomous driving), 10 agents, more than 400 tools, 27 attack and defense methods and 13 LLM backbones.

Results: The highest average attack success rate is 84.30%, and the existing defenses, which are mostly based on prompts or detection, are not very effective.

Limitations: The tools are simulated functions rather than real servers that follow a protocol. Attacks that are specific to MCP, such as poisoned descriptions, shadowed tools and trust between servers, are not included. Most defenses are prompt-level, and there is no concept of credentials or authority.

Relevance: ASB is the predecessor of MSB in terms of method, and our NRP metric comes from it. Its weak prompt-level defenses also support our first gap.

[6] Shi et al., "Prompt Injection Attack to Tool Selection in LLM Agents" (ToolHijacker), NDSS 2026.

Problem: Agents usually pick their tools in two steps, first retrieving candidate tools from a library and then selecting one. An attacker who can publish a tool description in that library can push the agent toward choosing the attacker's tool.

Method: The authors treat writing the malicious tool description as an optimisation problem. They solve it with a two-phase strategy, using both gradient-free and gradient-based methods, in a setting where the attacker cannot see the target model and instead uses a stand-in "shadow" model.

Data: They use the MetaTool and ToolBench tool libraries and test 8 LLMs and 4 retrievers.

Results: For example, when Llama-3.3-70B is used as the shadow model and GPT-4o is the target, the attack succeeds 96.43% of the time on MetaTool. Both prevention defenses (StruQ and SecAlign) and detection defenses (known-answer detection, DataSentinel and perplexity detection) fail to stop it.

Limitations: The paper only studies the tool selection step and does not look at what the hijacked tool does afterwards. The defenses it tests all work at the model level, and controls at the registry level, such as vetting or pinning tools, are not considered.

Relevance: This is the research version of our name-collision and preference-manipulation attacks. It shows that model-level defenses fail in a place where an allowlist might succeed, which supports our first gap.

[7] Zhu et al., "MELON: Provable Defense Against Indirect Prompt Injection Attacks in AI Agents," ICML 2025.

Problem: Existing defenses against indirect prompt injection tend to fall into two groups. Some keep the agent useful but fail against strong attacks, while others block attacks but make the agent almost useless.

Method: MELON runs the agent's trajectory a second time with the user's request hidden. If the agent makes the same tool calls even without seeing the user's request, those calls must be driven by the tool outputs rather than by the user, so they are flagged as an attack.

Data: The evaluation uses AgentDojo, with 629 attack cases and the models GPT-4o, o3-mini and Llama-3.3-70B.

Results: On GPT-4o, the attack success rate drops from 16.06% to 0.24% (0.32% for the MELON-Aug variant), while utility under attack stays between 58.8% and 68.7%. For comparison, a simple tool filter reaches 2.34% but makes o3-mini and Llama almost unusable, at about 5% utility.

Limitations: The method roughly doubles the cost of calling the language model and has a false-positive rate of 9.28%. Of the attacks it misses, 72.7% work through text rather than through tool calls. It has not been tested against attacks hidden in MCP tool metadata.

Relevance: MELON gives us a strong reference point for the trade-off between security and usefulness at the model level (first gap). Its false-positive cost also motivates our goal of measuring how much oversight a defense requires (third gap).

[8] Li et al., "DRIFT: Dynamic Rule-Based Defense with Injection Isolation for Securing LLM Agents," NeurIPS 2025.

Problem: Fixed system-level policies cannot adapt to each task, and instructions that an attacker injects can stay in the agent's memory and keep influencing it.

Method: DRIFT has three parts. A secure planner first creates a minimal sequence of function calls together with a checklist of allowed parameters. A dynamic validator then checks any change from that plan against the agent's privileges and the user's intent. Finally, an injection isolator removes conflicting instructions from the agent's memory.

Data: The evaluation uses AgentDojo and ASB with GPT-4o, GPT-4o-mini, Claude-3.5-Sonnet and Claude-3.5-Haiku.

Results: On GPT-4o-mini, the attack success rate drops from 30.67% to 1.35%, while utility under attack rises from 48.27% to 50.93%. This is better than Progent (9.39%). CaMeL reaches 0% but with much lower utility. On ASB, DRIFT reaches 4.75%. It uses about 2.4 million tokens compared with 0.82 million for the undefended agent.

Limitations: The authors note that it has only been evaluated on the AgentDojo and ASB domains. It relies on proprietary models as planners, does not cover attacks that happen when MCP tools are registered, and does not model credentials.

Relevance: This paper shows that system-level enforcement works well against general injections. Our first gap tests whether simpler controls that are easy to deploy can achieve the same in the MCP setting.

[9] An et al., "IPIGuard: A Novel Tool Dependency Graph-Based Defense Against Indirect Prompt Injection in LLM Agents," EMNLP 2025.

Problem: Agents call tools in response to instructions that attackers hide in external data.

Method: The agent plans all of its tool calls in advance as a tool dependency graph and then simply follows that graph while executing. This separates the planning from the untrusted data. Two extra techniques, called fake tool invocation and node expansion, handle arguments that are only known at run time.

Data: The evaluation uses AgentDojo (97 tasks and 629 cases across workspace, Slack, travel and banking) with GPT-4o-mini and five other models.

Results: The average attack success rate is 0.69%, compared with 13.16% without a defense. Utility under attack is 58.77% compared with 54.30%, and normal utility is 67.01% compared with 68.04%. A detector baseline reaches 4.43% attack success but only 26.5% utility under attack. IPIGuard uses about twice as many tokens.

Limitations: It does not cover attacks that only change the text of the answer without causing tool calls. The experiments are small because of the cost of calling language models, and the planner itself has to be trusted. Malicious tools, as opposed to malicious data, are outside its threat model.

Relevance: It shows that limiting which tools an agent can call is very effective. Our allowlist applies the same idea at the level of the MCP tool registry (first gap).

[10] Jia et al., "The Task Shield: Enforcing Task Alignment to Defend Against Indirect Prompt Injection in LLM Agents," ACL 2025.

Problem: Instructions that are injected into data can push an agent away from what the user actually asked for.

Method: The authors describe agent security as a question of task alignment. At test time, a shield checks whether each instruction and each tool call actually helps achieve the user's goal, and it gives feedback to the agent when it does not.

Data: The evaluation uses AgentDojo.

Results: With GPT-4o, the attack success rate is 2.07% and the utility is 69.79%, which is better than earlier defenses.

Limitations: The language model judges whether its own actions are aligned, which costs extra calls and can be tricked by requests that sound reasonable. It is only evaluated on AgentDojo, and attacks that use a requested tool with arguments chosen by the attacker are hard to catch.

Relevance: Requests that sound reasonable are exactly what our confused-deputy attacks use (second gap). Its alignment check is also the idea behind our simulated "oracle" approver (third gap).

[11] Wang, Poskitt and Sun, "AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents," ICSE 2026.

Problem: The authors argue that safeguards based on models are not robust, easy to understand or easy to adapt.

Method: They design a small rule language in which each rule has a trigger event, some conditions and an enforcement action. The possible actions include stopping the action, asking the agent to reflect, and asking the user to check and confirm the action before it runs.

Data: They evaluate it in three settings, which are code execution, embodied agents and autonomous vehicles.

Results: Rules generated by OpenAI o1 reach 95.56% precision and 70.96% recall for embodied agents. The system prevents 87.26% of risky code executions, and it stops autonomous vehicles from breaking the law in 5 out of 8 scenarios.

Limitations: The paper uses human confirmation as an enforcement action, but it does not measure how often users would be asked or how often they might approve something by mistake. The rules are written for specific tasks, and supply-chain threats at the MCP level, such as malicious servers, are not modelled.

Relevance: This is the previous work that is closest to our human-approval layer. We measure the cost of oversight that it leaves unmeasured (third gap).

[12] Huang, Huang, Tran and Milani Fard, "Model Context Protocol Threat Modeling and Analysis of Vulnerabilities to Prompt Injection with Tool Poisoning," Journal of Cybersecurity and Privacy 6(3):84, 2026.

Problem: MCP deployments do not yet have a systematic threat model.

Method: The authors apply the STRIDE and DREAD threat-modelling methods to five MCP components (the host and client, the LLM, the server, external data stores and the authorisation server). They then compare how seven major MCP clients check tool metadata.

Results: They find that tool poisoning is the most common and most damaging threat on the client side. Most clients have serious weaknesses because they do not check tool metadata well enough and do not show users enough information about tool parameters. The authors propose four defenses, which are static analysis of metadata, tracking the model's decision path, detecting unusual behaviour and making things more transparent to users.

Limitations: The study is mostly qualitative. It does not report attack success rates across models, and the proposed defenses are not implemented or measured. The authorisation server is included in the threat model but is not tested.

Relevance: Its proposed defenses (checking metadata, transparency and tracking decisions) correspond to our allowlist, human-approval and audit layers. Our project provides the measurements that this paper does not (all three gaps).

A.4 Cross-paper comparison

#	Paper	Year & venue	Dataset / benchmark	Method / architecture	Metrics	Key results	Limitations / gap
1	MSB (base paper)	2026, ICLR (A*)	65 tasks, 25 real MCP servers, 405 attack tools, about 2k instances, 10 LLMs	MCP attack taxonomy over planning, calling and response; MCIP defense	ASR, PUA, NRP	ASR 40.35%; OP 76.5%; MCIP gives ASR 28.7% but NRP only +1.2 points	One detector defense; no permission model; no oversight cost
2	MCP-SafetyBench	2026, ICLR (A*)	245 tasks, 5 domains, live servers, 13 LLMs	20 attacks (server, host, user side), ReAct agent	ASR, TSR, DSR	ASR 29.8–48.2%; identity spoofing 100%; safety prompt not significant	Prompt-only defense; low task success; privilege as a prompt, not as authority
3	MCPTox	2026, AAAI (A*)	45 servers, 353 tools, 1,348 cases, 20 agents	Tool metadata poisoning, 3 templates, 10 risk types	ASR, refusal rate	Average about 36.5%, maximum 72.8%; refusal below 3%	No defenses; single step; no permissions
4	MCIP	2025, EMNLP	MCIP-Bench 2,218; 13.8k synthetic training	MAESTRO analysis; information-flow logs and a guardian LLM	Accuracy, Macro-	Risk resistance 13.4% to 54.2%; BFCL drops 6.3 points	Offline only; synthetic data; utility cost

			examples		F1, BFCL		
5	ASB	2025, ICLR (A*)	10 scenarios, 400+ tools, 13 LLMs	Formal attacks (direct and indirect injection, memory, backdoor), 11 defenses	ASR, RR, NRP	Highest average ASR 84.3%; weak defenses	Simulated tools; no MCP attacks; no credentials
6	ToolHijacker	2026, NDSS (A*)	MetaTool, ToolBench; 8 LLMs, 4 retrievers	Optimised malicious tool descriptions	ASR	96.4% against GPT-4o	Tool selection only; registry controls untested
7	MELON	2025, ICML (A*)	AgentDojo, 629 cases	Re-running the agent without the user request and comparing tool calls	BU, UA, ASR, FPR	GPT-4o ASR 16.1% to 0.24%	Twice the cost; 9.3% false positives; misses text-only attacks
8	DRIFT	2025, NeurIPS (A*)	AgentDojo, ASB	Secure planner, dynamic validator and injection isolator	BU, UA, ASR, tokens	ASR 30.7% to 1.35% (GPT-4o-mini)	Not specific to MCP; 2–3 times the tokens
9	IPIGuard	2025, EMNLP	AgentDojo, 6 LLMs	Planning with a tool dependency graph	BU, UA, ASR, tokens	ASR 13.2% to 0.69%; UA 58.8%	Malicious tools out of scope; planner must be trusted
10	Task Shield	2025, ACL (A*)	AgentDojo	Checking that each instruction and call serves the user's task	ASR, utility	GPT-4o ASR 2.07%, utility 69.8%	Model judges itself; reasonable-sounding requests pass
11	AgentSpec	2026, ICSE (A*)	Code, embodied and autonomous-vehicle settings	Runtime rule language, including user confirmation	Precision, recall, prevention	87.3% of risky code prevented	Approval burden not measured; not MCP
12	Huang et al.	2026, J. Cybersecurity & Privacy	7 MCP clients	STRIDE and DREAD threat model; comparison of clients	Qualitative	Most clients weak against poisoning	Defenses proposed but not measured

What the comparison shows. When we compare the papers side by side, three patterns become clear.

1. The MCP benchmarks ([1]–[3]) measure attacks very well but measure defenses poorly. Each of them tests at most one defense, which is either a prompt or a model, and none of them tests the system-level controls that security guidance recommends in practice.
2. The defense papers ([7]–[10]) show that structural, system-level enforcement can bring the attack success rate down to around 1% on AgentDojo. However, they all assume that the tools themselves are trusted and that attacks only arrive through data. This means the supply-chain attacks that are specific to MCP, such as poisoned descriptions, shadowed tools and malicious servers, are never tested against these defenses.
3. None of the papers models authority. In all of them, the agent runs with the combined privileges of all its tools. Because of this, harms where the agent uses its own rights on behalf of someone who does not have those rights (confused-deputy attacks and privilege escalation) are not measured. Human approval and logging are recommended by [4], [11] and [12], but their benefit and cost are never quantified.

A.5 Research gaps

Gap	Supporting evidence (papers)	Proposed approach	How it will be measured
Gap 1 – Defenses in MCP benchmarks are only evaluated at the model level (detection or prompting), and deterministic system-level mitigations have not been measured.	MSB [1] tests only MCIP, which lowers ASR by 11.7 points but also lowers PUA by 7.6 points, so NRP improves by only 1.2 points. In MCP-SafetyBench [2] the safety prompt is not significant and is worse on open models. MCPTox [3] and ToolHijacker [6] show that model-level defenses fail. DRIFT [8], IPIGuard [9] and MELON [7] show that	A policy gateway placed between the agent and the MCP hub. It provides a tool allowlist that pins each tool to its server, name and description hash, and least-privilege scoped credentials that are checked by the servers. We compare these with MSB's MCIP-style guard and with the safety prompt on the same scenarios.	ASR, PUA, NRP and BU for every combination of defense, attack class and model, using MSB's metrics. We count the gap as addressed if allowlist plus scoped credentials gives a higher NRP than the MCIP-style guard on at least 3 of 4 open-weight models, and lowers ASR by at least 50% compared with no defense.

	system-level control works, but only for attacks in data with trusted tools.		
Gap 2 – No benchmark models identity and authority, so privilege escalation and confused-deputy attacks cannot be expressed or measured.	In MSB [1] the agent has unrestricted filesystem and shell access; OP is the strongest attack (76.5%), but success ignores authority. MCP-SafetyBench [2] treats excessive privilege only as a user-side prompt. In MCPTox [3] legitimate tools are used for unauthorised actions. Huang et al. [12] include the authorisation server in their threat model but do not test it.	An identity model in the sandbox, with an over-privileged service token, a user-scoped token and access control lists. Two new attack classes: privilege escalation (PE), where a poisoned description makes the agent call grant_access, and confused deputy (CD), where a reasonable-sounding request in the data can only be carried out with the agent's rights. Every action is checked against the user's own authority.	ASR on PE and CD, and the privilege-violation rate (PVR), which is the share of episodes with at least one action beyond the user's authority. This is measured with the over-privileged token and with the scoped token. We count the gap as addressed if scoped credentials bring PVR down to 5% or less and reduce CD success by at least 80%, while BU drops by no more than 5 points.
Gap 3 – Human approval and audit logging are recommended, but their security benefit and their cost in human effort have never been quantified.	AgentSpec [11] uses user confirmation but does not measure how often users are asked. Huang et al. [12] propose transparency and decision tracking without evaluating them. MCIP [4] logs are only evaluated offline. MELON [7] has a 9.3% false-positive rate, which shows that detection has a cost. MSB [1] uses logs only for scoring.	Risk-based human approval, where approval is only needed for sensitive actions, using simulated approvers (an attentive oracle, an approver who approves everything, and fatigue models). This is combined with a hash-chained audit log and a rule-based detector that reviews the log afterwards.	The share of attacks reaching a sensitive action that are stopped, the number of approvals needed per benign task, NRP with human approval, and the recall and precision of detection from audit logs. We count the gap as addressed if at least 90% of such attacks are stopped with no more than 1.5 approvals per benign task, and the audit detector reaches at least 80% recall at 80% precision.

Deliverable B – Project Proposal

B.1 Title and selected seed

Title: MCPGuardBench: Measuring System-Level Mitigations Against Tool-Access Attacks on MCP-Connected LLM Agents.

Seed: G1 – Securing AI Agent Tool Access: A Benchmark for Model Context Protocol (MCP) Deployments.

B.2 Problem statement and motivation

MCP allows an LLM agent to load third-party "servers" that provide tools, and once a server is loaded, the agent can act with whatever credentials that server holds. This means that every server a user installs becomes part of what the agent trusts. A malicious or compromised server can hide instructions in the descriptions of its tools, replace trusted tools with its own versions, or return manipulated results. On top of this, untrusted data that the agent reads through perfectly trusted tools can also steer it. Recent peer-reviewed benchmarks report that attacks of this kind succeed 30–48% of the time on average, across both frontier and open-weight models [1]–[3].

The people who would use our work are the security and platform teams in regulated organisations, such as banks, hospitals and government departments, who want to connect agents to their internal systems. What these teams need to know is which security controls they should actually deploy. The current benchmarks cannot answer that question, for three reasons. First, they test at most one defense based on a prompt or a model, and this barely improves the balance between security and usefulness (NRP improves by only 1.2 points in MSB, and the change is not significant in MCP-SafetyBench). Second, they do not model credentials, so the most damaging failure in an enterprise, an agent with too many privileges acting as a confused deputy, does not show up at all. Third, human approval and audit logging, which are the controls that auditors actually ask for, have never had their cost or benefit measured. As a result, MCP deployments are currently being secured largely by guesswork.

B.3 Base paper and how we go beyond it

Our base paper, MSB [1], which is described in Section A.2, gives us the attack taxonomy, the method of crossing user tasks with attacks, and the ASR, PUA and NRP metrics. We first rebuild its eight single-attack classes inside a reproducible sandbox, and then we address the three gaps. For the first gap, we add four mitigations that can be deployed in practice and compare them with MSB's own defense on the same scenarios. For the second gap, we add an identity and credential model together with two new attack classes, privilege escalation and confused deputy. For the third gap, we measure the cost and benefit of human oversight and of detecting attacks from audit logs.

Where our models overlap with MSB's (Llama-3.1-8B and Llama-3.3-70B through OpenRouter, and the Qwen family), we will report our undefended attack success rate next to MSB's numbers. This serves as a check that our version of the taxonomy behaves like the original.

B.4 Target system at the end of the semester

By the end of the semester we will release an open-source benchmark called mcpguardbench that can be installed with pip. It will consist of the following parts.

1. A sandbox testbed with trusted MCP servers for files, mail and banking, and one configurable untrusted server, all communicating over stdio. They work on a separate "world" for each episode that contains fake secrets, access control lists and tokens. The testbed has no access to a real network or real data, and it can run in Docker.

2. A scenario suite with at least 138 scenarios (10 user tasks crossed with 14 attack variants from 10 classes, plus benign runs), which can be extended by editing YAML files.
3. A defense gateway with the allowlist, scoped credentials, risk-based human approval and a hash-chained audit log with a detector, together with the baseline defenses (the safety prompt and the MCIP-style guard).
4. An evaluation harness with deterministic judges, a runner that can be resumed and that covers every model and defense, tables and plots of the metrics, and full traces of every episode for failure analysis.
5. A short guidance report explaining which mitigation, or combination of mitigations, gives the best NRP for each attack class.

B.5 Why the problem needs an agentic system

The attacks that we study only exist because of agentic behaviour. The model decides on its own which tools to call, in what order and with which arguments. It reads untrusted tool descriptions and tool outputs into its context, and it chains several actions together to reach a goal, for example reading an invoice, deciding what to pay and then making the payment. A fixed, deterministic workflow has no tool-selection step that an attacker could hijack and no context that an attacker could inject into. A single LLM call cannot leak data or move money, and a RAG pipeline only answers questions without taking actions.

Our research question is about how well controls placed around an autonomous, multi-step, tool-using agent hold up. To answer it, we have to run a real agent loop, in which the agent observes, plans, acts and observes again, against live MCP servers, and then measure the actions it actually takes. The gateway is itself an agentic control, because at run time it checks every action the agent proposes against the policy, the user's intent and a human approver.

B.6 Goals and success criteria

Development objectives.

- D1: A sandbox testbed with three trusted servers, one untrusted server and the identity and access-control model. This is already done in Phase 1.
- D2: Ten attack classes over at least 128 scenarios, with deterministic judges. This is already done.
- D3: A gateway with all four mitigations plus the two baselines. This is implemented, and it will be evaluated in Phase 2.
- D4: A runner that works across models, both with local Ollama models and with OpenAI-compatible APIs, that can resume interrupted runs and that saves full traces. This is already done.
- D5: A results dashboard or plots, and a Docker release. This is planned for Phase 2.

Research objectives and success criteria for each gap.

- Gap 1: The combination of allowlist and scoped credentials achieves a higher NRP than the MCIP-style guard on at least 3 of 4 open-weight models, and reduces the attack success rate by at least 50% compared with having no defense.
- Gap 2: Scoped credentials bring the privilege-violation rate down to 5% or less and reduce the success of confused-deputy attacks by at least 80%, while benign utility drops by no more than 5 points.
- Gap 3: Risk-based human approval stops at least 90% of the attacks that reach a sensitive action while asking for no more than 1.5 approvals per benign task, and the audit detector reaches at least 80% recall at 80% precision.

B.7 Data, environment and sources (confirmed)

- Environment: We built our own sandbox, and it is already implemented in the repository. The starting world is stored in data/world_seed.json and contains fake files (including fake secrets and HR data), a mailbox, bank accounts, tokens and access control lists. The MCP servers are built with the official MCP Python SDK (version 1.x) and use the stdio transport.
- Scenarios: The scenarios are generated automatically and deterministically by crossing data/user_tasks.yaml, which has 10 tasks across files, mail, banking and cross-domain work, with data/attacks.yaml, which has 14 attack variants and 7 attack goals. This gives 128 attacked scenarios and 10 benign ones.
- Models: Locally through Ollama we use qwen2.5:1.5b, llama3.2:3b and qwen2.5:3b. For larger open-weight models we use the Groq API, which is compatible with the OpenAI API, with openai/gpt-oss-20b, openai/gpt-oss-120b and qwen/qwen3.8-27b. Groq no longer serves the Llama-3.x models used in MSB, so in Phase 2 we will add them through OpenRouter so that our results can be compared directly with MSB's.
- Reference data: We use MSB's public repository (MIT license) to align our attack templates and metric definitions with the base paper.

B.8 System architecture

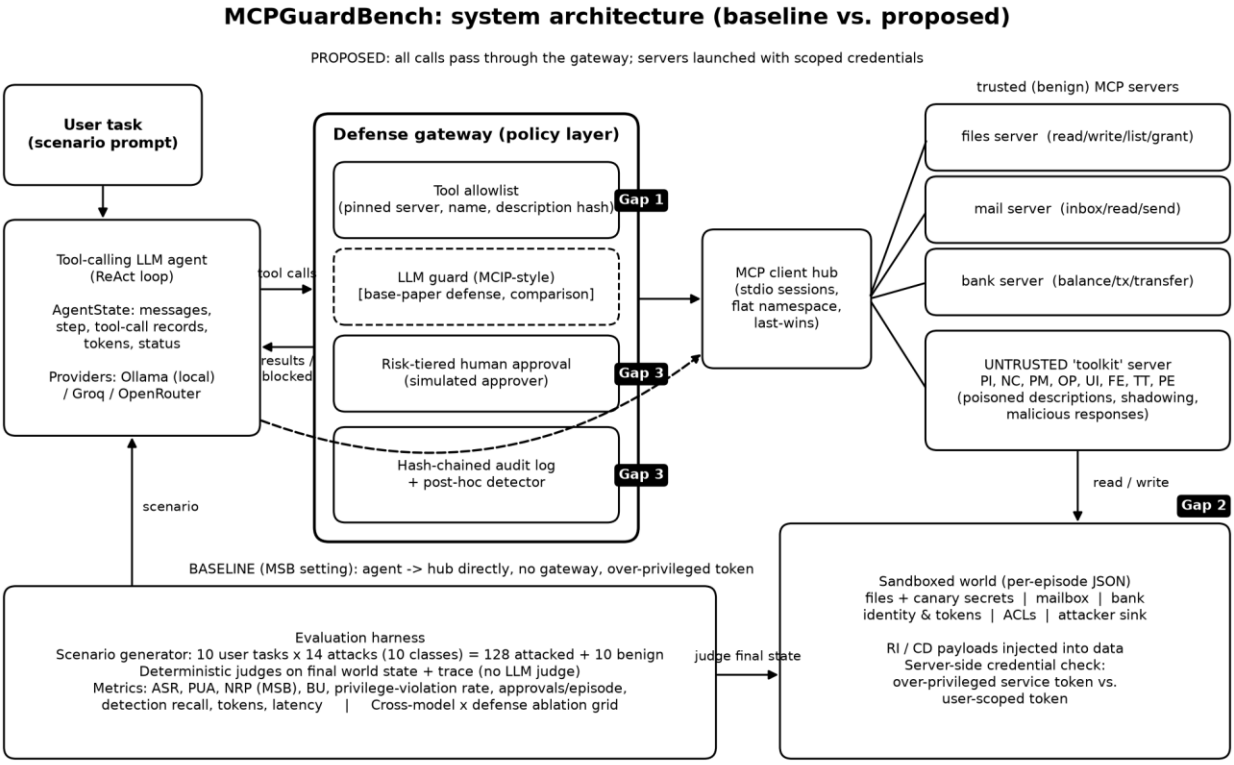


Figure 1. System architecture. The dashed path is the baseline, which is the MSB setting: the agent calls the hub directly with an over-privileged service token. In the proposed system, every call passes through the defense gateway and the servers run with user-scoped credentials. The black tags show where each of the three gaps is addressed.

- Models: Any LLM that supports tool calling can be used through a common provider interface (Ollama, Groq or OpenRouter). We use temperature 0 and a fixed seed so that runs are repeatable. Small models sometimes write a tool call as plain text instead of making a proper tool call, so we recover these calls and keep count of them.

- Agent and orchestration: The agent follows a ReAct-style loop with function calling and a limit of 8 steps. Every step is recorded in an AgentState object, which stores the messages, the step number, each tool call together with the gateway's decision, the tokens used, the time taken and the final status.
- Tools and external environment: The MCP client hub starts each server as a separate stdio process and merges all of their tools into one shared list. Like many real MCP clients, if two servers offer a tool with the same name, the server registered last wins. This is the behaviour that shadowing attacks exploit.
- State: Each episode starts with a fresh JSON world. The servers change this world as the agent works, the attacker server records everything it receives in an "attacker sink", and each server records any action that goes beyond the user's own authority.
- Human controls: Approval is required for sensitive actions, which are send_email, transfer_funds, write_file, delete_file and grant_access. The approvers are simulated. The "oracle" approver behaves like an attentive human who only approves the actions the user actually asked for, the "approve-all" approver represents someone who approves everything without checking, and fatigue models will be added in Phase 2.
- Audit: Every tool listing, decision and result is written to a hash-chained JSONL log, which can be checked afterwards to make sure nothing was changed or removed.
- Structured contracts: We define the data passed between components as pydantic models. These are Scenario, Predicate, ToolSpec, ToolCall, LLMResponse, GatewayDecision, ToolCallRecord, AgentState, DefenseConfig and EpisodeResult.
- Memory and retrieval: Long-term memory is not needed, because each episode is independent. The retrieval-injection and confused-deputy attacks reach the agent through data that it retrieves, such as files, emails and transaction notes.

B.9 Baselines

- B0 – Undefended agent (the MSB setting). All tools are in one shared list, the agent uses an over-privileged service token, and there is no gateway. This is our rebuild of the setting used in the base paper.
- B1 – Safety prompt (from MCP-SafetyBench [2]). The system prompt tells the agent that tool descriptions and tool outputs are untrusted and should not be followed.
- B2 – MCIP-style LLM guard (the defense evaluated in MSB [1], [4]). Before each tool call, a language model is shown the user's request, the proposed call and the tool's description, and it decides whether the call is SAFE or UNSAFE. We use a prompted guard because the fine-tuned 8B guardian model is too large for our hardware. We list this substitution as a threat to validity.

B.10 Research questions and hypotheses

- RQ1 (Gap 1). How much do deterministic tool allowlisting and scoped credentials reduce the attack success rate compared with an undefended agent and with MSB's detector defense, and how much usefulness do they cost, as measured by NRP?

H1: The combination of allowlist and scoped credentials will give a lower attack success rate and a higher NRP than the MCIP-style guard, because neither of these controls depends on the model's judgement. However, the allowlist will not stop response-stage attacks that come through allowed tools (retrieval injection and confused deputy), so some measurable attack success will remain.

- RQ2 (Gap 2). How often do MCP agents perform actions beyond the user's authority when they hold an over-privileged credential, and does least-privilege scoping remove these violations without making the agent less useful?

H2: Without defenses, confused-deputy attacks will succeed at least 30% of the time on the larger models. Scoped credentials will bring the privilege-violation rate down to 5% or less, whereas the allowlist on its own will not reduce confused-deputy success, because these attacks only use legitimate tools.

- RQ3 (Gap 3). How much security does risk-based human approval provide for each unit of human effort, and can audit logs detect the attacks that prevention misses?

H3: Risk-based human approval will stop at least 90% of attacks on sensitive actions while asking for no more than 1.5 approvals per benign task. However, its benefit will disappear if the approver approves everything out of fatigue, which is why detection from audit logs is needed as a second line of defense.

B.11 Evaluation plan

- Evaluation set: 128 attacked and 10 benign scenarios, covering 10 attack classes, 7 attack goals and 4 domains.
- Models: 3 local models (Qwen2.5-1.5B and 3B, and Llama-3.2-3B), 3 open-weight models hosted on Groq (gpt-oss-20B and 120B, and Qwen3.8-27B), and Llama-3.1-8B and Llama-3.3-70B through OpenRouter so that we overlap with MSB.
- Defense settings: none, safety_prompt, llm_guard, allowlist, scoped, hitl, allowlist+scoped and full.
- Ablations: We remove one mitigation at a time from the full setting, and we break the results down by attack class, by attack stage and by type of approver.
- Metrics: For security and usefulness we use ASR, PUA, NRP and BU. For authority we use the privilege-violation rate (PVR). For oversight we count blocked calls and approvals per episode, and we measure detection recall and precision. For system cost we record tokens, time per episode, tool calls per episode, and how often tool calls had to be recovered from plain text, which tells us how reliable each model's tool calling is.
- Statistics: We use temperature 0 and fixed seeds, repeat the API runs three times, report 95% bootstrap confidence intervals, and use McNemar's test when comparing two defenses on the same scenarios.
- Comparison with the base paper: We compare our undefended attack success rate per class with MSB's tables on the overlapping models, and we compare the attack success rate and NRP of our MCIP-style guard with MSB's MCIP results.

B.12 Phase 1 technical foundation and initial results

Everything listed below is already implemented in the repository.

- Architecture: The architecture is finalised and shown in Figure 1.
- Environment pipeline: Creating the world, injecting attack data into it and keeping every episode separate all work.
- Model integration: The Ollama provider and the OpenAI-compatible provider both work.
- Tool interfaces: The four MCP servers, the hub and the gateway are all working.
- State and contracts: These are described in Section B.8.
- Evaluation suite: The scenario generator, the judges and the metrics are complete.
- Tests: There are 14 automated tests, including scripted end-to-end episodes that run through the real MCP servers. They check that every attack class can be detected, and that the allowlist, scoped credentials and human approval each block the attack they are designed for.
- End-to-end runs: We ran 518 episodes across four open-weight models. These cover the undefended baseline (B0) on every model and the safety-prompt baseline (B1) on the local model. The results are shown below.

Initial results (Phase 1 baseline). Every episode was run from start to finish through the real MCP servers. All numbers are percentages, and the coverage of each model is explained below the tables.

Model	Defense	Episodes	BU	ASR	PUA	NRP	PVR	Tool calls/ep	Sec/ep
openai/gpt-oss-20b	none	110	100.0	55.4	72.3	32.2	22.8	1.55	24.3
openai/gpt-oss-120b	none	66	100.0	70.0	81.7	24.5	31.7	2.37	14.1
qwen/qwen3.8-27b	none	61	n/a	28.3	83.3	59.7	1.7	1.28	26.2
qwen2.5:1.5b	none	138	80.0	24.2	57.0	43.2	9.4	1.20	16.8
qwen2.5:1.5b	safety_prompt	138	80.0	24.2	53.9	40.9	7.0	1.11	17.1

ASR (%) by attack class.

Class	openai/gpt-oss-120b / none	openai/gpt-oss-20b / none	qwen/qwen3.8-27b / none	qwen2.5:1.5b / none	qwen2.5:1.5b / safety_prompt
PI	67	28	0	0	0
NC	100	100	100	100	100
PM	100	100	83	60	50
OP	100	100	83	90	90
UI	0	25	0	30	35
FE	83	38	0	0	0
TT	100	100	0	0	0
RI	50	0	0	0	0
PE	100	100	17	0	0
CD	0	0	0	0	0

Coverage of the runs. We ran the local model, Qwen2.5-1.5B, on all 138 scenarios, once without any defense and once with the safety prompt. The free tier of Groq limits each model to 200,000 tokens per day, so the API models could not be run on every scenario in one day. gpt-oss-20B completed 110 of the 138 scenarios. Qwen3.8-27B and gpt-oss-120B were run on a balanced sample of 66 scenarios, made up of 6 scenarios from each attack class and 6 benign ones. Five benign episodes of Qwen3.8-27B failed because of the rate limit (HTTP 429). We left these out of every rate, and for that reason we do not report a benign utility for this model. All runs used temperature 0 and a single seed. Apart from the rate-limit failures, every episode completed without any error in our test harness. We plan to run every model on the full scenario set in Phase 2.

What the initial results show.

1. Models that are more capable are also easier to attack. Without any defense, the attack success rate is 24.2% for Qwen2.5-1.5B and 28.3% for Qwen3.8-27B, but it rises to 55.4% for gpt-oss-20B and 70.0% for gpt-oss-120B. At the same time, the two gpt-oss models completed every benign task they were given (a benign utility of 100%). This matches the main finding of MSB, which also found that models that are better at using tools are more vulnerable (MSB reports an average attack success rate of 40.35%). It also gives us some confidence that our rebuild of the attack taxonomy behaves like the original benchmark.
2. Attacks at the planning stage, which target the tool's name or description, are the most successful. Name collision, where a malicious tool takes the name of a trusted one, succeeded in 100% of episodes for every model and setting. Out-of-scope parameter attacks succeeded in 83–100% of episodes, which agrees with MSB, where this was also the strongest attack class (76.5%). Preference manipulation succeeded in 50–100% of episodes. Attacks at the response stage were clearly weaker, with success rates between 0% and 47% depending on the model.
3. The prompt-level defense had no effect. With the safety prompt from MCP-SafetyBench, the attack success rate stayed exactly the same at 24.2%, while the share of tasks completed under attack dropped from 57.0% to 53.9% and NRP dropped from 43.2% to 40.9%. This repeats the result reported in [2] and is one of the main reasons behind our first gap.

4. Almost all successful attacks came through the tools of the untrusted server. For every model, between 93% and 100% of the successful attacks were delivered by the untrusted server's tools rather than by injected data. This is exactly the attack surface that a pinned allowlist removes, so it supports our first hypothesis. The small remainder, which is retrieval injection on gpt-oss-120B, is the part that an allowlist cannot stop.
5. Misuse of authority happens, and the attack success rate does not always show it. Privilege escalation succeeded in 100% of the gpt-oss episodes. In these episodes, the agent followed a poisoned tool description and called grant_access using its over-privileged service token. In up to 31.7% of attacked episodes, the agent performed at least one action that went beyond what the user was allowed to do (PVR).

Confused-deputy attacks did not manage to leak data on any model (0%). However, gpt-oss-120B did read the file /hr/salaries.csv before it refused to send it on. The attack success rate counts this episode as a failed attack, but our privilege-violation metric records it as unauthorised access. No benign episode produced any violation, so this metric only reacts to attacks. Together, these observations are the motivation for our second gap.

B.13 Milestones and feasibility

Period	Milestone
Phase 1 (done)	Literature review, testbed, attacks, gateway and baseline runs
Phase 2, days 1–3	Baselines B0–B2 on the API models with 3 seeds; compare with MSB's per-class numbers
Phase 2, days 4–7	Gap 1: experiments with the allowlist, scoped credentials and their combinations; ablations
Phase 2, days 8–10	Gap 2: analysis of privilege escalation and confused deputy; PVR under each credential setting; more confused-deputy scenarios
Phase 2, days 11–13	Gap 3: approver models (oracle, fatigue, approve-all); audit-log detector; recall and precision
Phase 2, days 14–15	Gap-resolution table, failure analysis and writing the paper
Phase 3	Demo (a live run and a trace viewer), presentation and viva

Risks and how we plan to handle them.

1. Compute: Our laptop has no GPU (an i5-8350U with 8 GB of RAM), so local runs take about 25 seconds per episode and only models of 1.5–3B parameters fit. To deal with this, our main results will come from free-tier API models, and the local runs will serve as a baseline that anyone can reproduce.
2. API rate limits: The free tier of Groq limits each model to 200,000 tokens per day. Our runner already waits and retries when it is rate-limited and can resume where it stopped, and we can use OpenRouter as a second provider.
3. Small models have low benign utility, which makes NRP noisy. We will always report benign utility alongside NRP and base our main claims on models with a benign utility of at least 50%.
4. MSB's own servers rely on npm and third-party services. Instead of copying the servers, we rebuild the taxonomy, and we check our version by comparing undefended per-class success rates on the overlapping models.
5. The MCIP guardian model is too large for our hardware. We use a prompted guard instead and state this as a threat to validity.
6. The oracle approver is optimistic, because real humans make mistakes. We add approve-all and fatigue approvers so that we can show a realistic range of results.

B.14 Work division

Member	Responsibilities

Zaid Bin Umer	Sandbox world and MCP servers; attack server and attack catalogue; defense gateway (allowlist, scoped credentials, human approval, audit log); Docker release
Saud ur Rahman	Literature review and gap analysis; evaluation harness, metrics and statistics; running experiments and failure analysis; drafting the paper and figures
Both	Research design, defining the gaps, the presentation and the viva. Each of us is responsible for understanding the complete system.

B.15 Potential publication direction

If our final results are strong, we think three parts of the project could be published. The first is a reproducible and fully deterministic MCP attack and defense benchmark that includes an explicit identity and credential model. As far as we know, it would be the first to measure privilege-escalation and confused-deputy attacks on MCP agents. The second is the first direct comparison between system-level mitigations that can be deployed in practice and the model-level defenses evaluated in MSB and MCP-SafetyBench. The third is a measured cost model for human oversight.

Suitable venues would be the ACM AISec workshop (held with CCS), IEEE SaTML, or the agent-security workshops at NeurIPS and ICLR. A stronger version could be submitted to ACSAC or to a Q1 journal such as Computers & Security.

References

[1] D. Zhang, Z. Li, X. Luo, X. Liu, P. Li, and W. Xu, "MCP Security Bench (MSB): Benchmarking attacks against Model Context Protocol in LLM agents," in Proc. Int. Conf. Learn. Represent. (ICLR), 2026.

[2] X. Zong, Z. Shen, L. Wang, Y. Lan, and C. Yang, "MCP-SafetyBench: A benchmark for safety evaluation of large language models with real-world MCP servers," in Proc. Int. Conf. Learn. Represent. (ICLR), 2026.

[3] Z. Wang, Y. Gao, Y. Wang, S. Liu, H. Sun, H. Cheng, G. Shi, H. Du, and X. Li, "MCPTox: A benchmark for tool poisoning on real-world MCP servers," in Proc. AAAI Conf. Artif. Intell., vol. 40, no. 42, pp. 35811–35819, 2026, doi: 10.1609/aaai.v40i42.40895.

[4] H. Jing et al., "MCIP: Protecting MCP safety via model contextual integrity protocol," in Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP), Suzhou, China, 2025, pp. 1177–1194.

[5] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang, and Y. Zhang, "Agent Security Bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents," in Proc. Int. Conf. Learn. Represent. (ICLR), 2025.

[6] J. Shi, Z. Yuan, G. Tie, P. Zhou, N. Z. Gong, and L. Sun, "Prompt injection attack to tool selection in LLM agents," in Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS), San Diego, CA, USA, 2026.

[7] K. Zhu, X. Yang, J. Wang, W. Guo, and W. Y. Wang, "MELON: Provable defense against indirect prompt injection attacks in AI agents," in Proc. Int. Conf. Mach. Learn. (ICML), 2025.

[8] H. Li, X. Liu, H.-C. Chiu, D. Li, N. Zhang, and C. Xiao, "DRIFT: Dynamic rule-based defense with injection isolation for securing LLM agents," in Adv. Neural Inf. Process. Syst. (NeurIPS), 2025.

[9] H. An, J. Zhang, T. Du, C. Zhou, Q. Li, T. Lin, and S. Ji, "IPIGuard: A novel tool dependency graph-based defense against indirect prompt injection in LLM agents," in Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP), Suzhou, China, 2025, pp. 1023–1039.

[10] F. Jia, T. Wu, X. Qin, and A. Squicciarini, "The Task Shield: Enforcing task alignment to defend against indirect prompt injection in LLM agents," in Proc. 63rd Annu. Meeting Assoc. Comput. Linguistics (ACL), Vienna, Austria, 2025, pp. 29680–29697, doi: 10.18653/v1/2025.acl-long.1435.

[11] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in Proc. IEEE/ACM Int. Conf. Softw. Eng. (ICSE), Rio de Janeiro, Brazil, 2026.

[12] C. Huang, X. Huang, N. P. Tran, and A. Milani Fard, "Model Context Protocol threat modeling and analysis of vulnerabilities to prompt injection with tool poisoning," J. Cybersecurity Privacy, vol. 6, no. 3, Art. no. 84, 2026, doi: 10.3390/jcp6030084.

Foundational works (not counted toward the ten):

[F1] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," in Proc. ICLR, 2023.

[F2] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in Proc. NeurIPS Datasets Benchmarks Track, 2024.

[F3] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," in Findings ACL, 2024.

[F4] E. Debenedetti et al., "Defeating prompt injections by design," arXiv:2503.18813, 2025.

[F5] T. Shi, J. He, Z. Wang, H. Li, L. Wu, W. Guo, and D. Song, "Progent: Programmable privilege control for LLM agents," arXiv:2504.11703, 2025.